Intelligent Vision System for Attendance and Access Monitoring

---

1. Objective

The objective of this project is to build a robust **Intelligent Vision System** capable of identifying, tracking, and logging the movements of known personnel (e.g., Neeraj, Yashith) and unknown visitors at a monitored access point. The system must maintain a reliable state log (inside/outside) and support querying for historical movement data, duration of stay, and current status.

---

2. Requirements

2.1 Core Technology: Person Re-Identification (ReID)

The core functionality requires moving the system's capability from simple face presence detection to **Person Re-Identification**.

- **Component:** The fundamental difference is moving from "Face is present" to "Known Person X is present".
- **Input Data:** A database of **face encodings** for the three employees.
- **Goal:** Compare the detected face encoding to the database to get an ID.
- **Output:** A recognized person's name: **"Neeraj," "Yashith," or "Unknown"**.

2.2 Tracking and State Management

The system requires implementing a **simple state machine** and a database (like a CSV file or a simple SQL database) to record the events.

The **Tracking Logic** for a single camera at an office entrance/exit is as follows:

- If the last recorded action was **"ENTRY"**: The new action is recorded as **"EXIT"**.
- If the last recorded action was **"EXIT"** or empty: The new action is recorded as **"ENTRY"**.

2.3 Data Log (Database Schema)

Every time a recognized person passes the boundary, a new record must be saved.

Field Name	Description
Log ID	Unique identifier for the event.

Field Name	Description
Person ID	The recognized name/ID (e.g., Neeraj, Unknown_Visitor).
Timestamp	Exact date and time of the event (e.g., 2025-10-14 09:00:00).
Action	The logged state change: ENTRY or EXIT.

---

3. Methodology: System Phases

The development process is divided into three interconnected phases:

3.1 The Core Technology: Person Re-Identification (ReID)

This phase establishes the ability to reliably distinguish between known individuals and unknowns based on their face encodings.

---

i> VS Code and Python Setup

1. **Install VS Code:** Download and install Visual Studio Code.
2. **Install Python:** Download and install **Python 3.8 or later** (ensure you check the box that says "Add Python to PATH" during installation).
3. **Install VS Code Extensions:** Open VS Code and install the following extensions from the Extensions view (Ctrl+Shift+X):
    - **Python** (Microsoft)
    - **Pylance** (Microsoft)

---

ii> Environment and Library Installation

The system requires several powerful libraries for data handling, face recognition, and video processing.

1. **Open Terminal:** In VS Code, open a new Terminal (`Ctrl+``).
2. **Install Libraries:** Run the following command to install all required libraries.
    - **Note:** The face_recognition library relies on the C++ library dlib, which can sometimes be tricky to install. If the command below fails, you might need to install Visual C++ Build Tools (on Windows) or cmake/boost (on Linux/macOS) first.

Bash

pip install opencv-python face-recognition numpy pandas python-dateutil ultralytics

3. **Install Windows Audio Dependency (Optional):** If you are on Windows and want the audible security alert for the unknown visitor, run:

Bash

pip install winsound

(If you are not on Windows, the winsound calls in the Python script will simply fail silently, and you don't need this step.)

---

iii> Project File Preparation

You need to place all the necessary Python files and the employee image in one dedicated project folder.

1. **Create Project Folder:** Create a new folder on your desktop (e.g., Face_Attendance).
2. **Place Files:** Ensure the following files are inside this folder:
    - **attendance_app.py** (The main camera script)
    - **report_tool.py** (The query script)
    - **neeraj.jpg** (The employee photo for encoding)
3. **Initial Data File:** The attendance_log.csv file will be created **automatically** when you run attendance_app.py for the first time.

---

iv> Hardware Check

1. **Webcam Access:** Ensure your computer's webcam is working and is not being used by another application (like Zoom or Teams), as this will prevent cv2.VideoCapture(0) from accessing it.
2. **Camera Index:** If your laptop has an integrated camera and you use an external USB camera, the index 0 might be wrong. If the webcam does not open, try changing the camera index in attendance_app.py from 0 to 1 or 2

Sample python code file (attendance.py) :

```
import cv2
import face_recognition
import numpy as np
import csv
from datetime import datetime
import os
import time
import pandas as pd
from typing import List, Dict, Tuple, Any
import winsound # For audio alerts (Windows only, comment out or change if not on Windows)
```

```
# --- 1. CONFIGURATION ---
LOG_FILE = 'attendance_log.csv'
EMPLOYEE_PHOTO_PATH = 'neeraj.jpg' # CHANGE THIS to your photo file!
COOLDOWN_SECONDS = 5
FRAME_SCALE = 0.25
UNKNOWN_VISITOR_ID = "Unknown_Visitor" # ID used in the log for all unknown faces
TOLERANCE = 0.5  # Standard is 0.6, we use 0.5 for stricter access control
FRAME_THICKNESS = 2
FONT_THICKNESS = 2
MODEL = "hog"   # hog for cpu and cnn for gpu

# --- 2. SETUP: KNOWN FACES ---

def load_known_faces() -> Tuple[List[Any], List[str]]:
    """Loads known face encodings from files."""
    known_face_encodings = []
    known_face_names = []

    # Example for one person (Neeraj)
    try:
        image = face_recognition.load_image_file(EMPLOYEE_PHOTO_PATH)
        encoding = face_recognition.face_encodings(image)

        if encoding:
            known_face_encodings.append(encoding[0])
            known_face_names.append("Neeraj")
            print(f"Loaded encoding for Neeraj.")
        else:
            print(f"Error: No face found in {EMPLOYEE_PHOTO_PATH}. Check photo alignment.")

    except FileNotFoundError:
        print(f"Error: Employee photo not found at {EMPLOYEE_PHOTO_PATH}. Please check the
path.")
    except Exception as e:
        print(f"Could not load or process employee photo: {e}")

    # Add more employees here if necessary

    return known_face_encodings, known_face_names

# --- 3. LOGGING FUNCTIONS (Robust) ---

def initialize_log():
    """Creates the log file with headers if it doesn't exist."""
    if not os.path.exists(LOG_FILE):
        with open(LOG_FILE, 'w', newline='') as f:
            writer = csv.writer(f)
            writer.writerow(["Timestamp", "Person_ID", "Action"])
            print(f"Created new log file: {LOG_FILE}")

def get_last_action(name: str) -> str:
```

```
    """Reads the log to find the last recorded action for a person."""
    initialize_log()
    try:
        df = pd.read_csv(LOG_FILE)

        if df.empty or len(df) == 0:
            return "EXIT" # Default: Person is OUTSIDE

        # Ensures correct chronological order
        df['Timestamp'] = pd.to_datetime(df['Timestamp'], errors='coerce')
        df.dropna(subset=['Timestamp'], inplace=True) # Remove corrupt rows

        person_log = df[df['Person_ID'] == name]

        if person_log.empty:
            return "EXIT" # If person has no log entries, they are OUTSIDE

        # Get the last action chronologically
        last_action = person_log.sort_values(by='Timestamp', ascending=False)['Action'].iloc[0]

        return last_action

    except Exception as e:
        print(f"Warning: Error reading log for status ({e}). Assuming EXIT.")
        return "EXIT"

def log_attendance(name, action):
    """Appends a new entry/exit record to the CSV log."""
    now = datetime.now()
    timestamp = now.strftime("%Y-%m-%d %H:%M:%S")

    with open(LOG_FILE, 'a', newline='') as f:
        writer = csv.writer(f)
        writer.writerow([timestamp, name, action])

    print(f"[LOGGED] {timestamp} - {name} has {action}.")

# --- 4. MAIN EXECUTION ---
known_face_encodings, known_face_names = load_known_faces()
initialize_log()

# Initialize webcam
video_capture = cv2.VideoCapture(0)

# Variables to prevent logging the same person multiple times per second
last_logged_time: Dict[str, float] = {}

while True:
    ret, frame = video_capture.read()
    if not ret:
        print("Error: Could not read frame from camera.")
```

```
        break

    # Fix for potential 4-channel BGRA frame bug
    if frame.shape[2] == 4:
        frame = cv2.cvtColor(frame, cv2.COLOR_BGRA2BGR)

    # Resize frame for faster processing and convert BGR to RGB
    small_frame = cv2.resize(frame, (0, 0), fx=FRAME_SCALE, fy=FRAME_SCALE)
    rgb_small_frame = cv2.cvtColor(small_frame, cv2.COLOR_BGR2RGB)

    face_locations = face_recognition.face_locations(rgb_small_frame)
    face_encodings = face_recognition.face_encodings(rgb_small_frame, face_locations)

    for face_encoding, (top, right, bottom, left) in zip(face_encodings, face_locations):
        name = "Unknown"
        action = "None"

        # Match face
        matches = face_recognition.compare_faces(known_face_encodings, face_encoding)
        face_distances = face_recognition.face_distance(known_face_encodings, face_encoding)
        best_match_index = np.argmin(face_distances)


        current_time = time.time()

        if matches[best_match_index]:
            # --- Known Person Logic ---
            name = known_face_names[best_match_index]
            log_id = name

            last_time = last_logged_time.get(log_id, 0)

            # Get the actual last action from the log file (the person's current status)
            current_status = get_last_action(log_id)
            action = current_status # Set initial action for display (current status)

            # 1. Check for cooldown
            if (current_time - last_time) > COOLDOWN_SECONDS:

                # 2. Determine the action to be logged (The opposite of the current status)
                if current_status == "ENTRY":
                    new_action = "EXIT"
                else:
                    new_action = "ENTRY"

                # 3. Log the event
                log_attendance(log_id, new_action)
                last_logged_time[log_id] = current_time
                action = new_action # CRITICAL FIX: Update 'action' to the newly logged action for
immediate display
            else:
```

```
            # If still in cooldown, action remains the current_status retrieved earlier
            pass

        # Set display colors based on the current determined status/action
        color = (0, 255, 0) if action == "ENTRY" else (0, 0, 255) # Green for ENTRY, Red for EXIT
        status_text = f"{name} ({action})"

    else:
        # --- Unknown Person Logic ---
        log_id = UNKNOWN_VISITOR_ID
        name = "Unknown"

        last_time = last_logged_time.get(log_id, 0)
        # Get the actual last action from the log file (the unknown visitor's current status)
        current_status = get_last_action(log_id)
        action = current_status # Set initial action for display (current status)

        # 1. Check for cooldown
        if (current_time - last_time) > COOLDOWN_SECONDS:

            # 2. Determine the action to be logged
            if current_status == "ENTRY":
                new_action = "EXIT"
            else:
                new_action = "ENTRY"

            # 3. Log the event using the generic ID
            log_attendance(log_id, new_action)
            last_logged_time[log_id] = current_time
            action = new_action # CRITICAL FIX: Update 'action' to the newly logged action for
immediate display

            # ============================================
            # 🛑 WARNING SYSTEM: TRIGGER ALERT ON ENTRY
            # ============================================
            if action == "ENTRY": # Check the NEWLY logged action
                # Visual: Print a big warning to the console
                print("\n" + "="*50)
                print(f"!!! SECURITY ALERT !!! UNKNOWN VISITOR DETECTED AT
{datetime.now().strftime('%H:%M:%S')}")
                print("="*50 + "\n")

                # Audible: Play a simple sound (Windows-specific)
                try:
                    winsound.Beep(1000, 500)
                except Exception:
                    pass
            # ============================================

        # Set display colors (Red for unknown alert)
        color = (0, 0, 255)
```

```
            status_text = f"!!! UNKNOWN ({action}) !!!"

        # Scale back up the face coordinates
        top, right, bottom, left = [int(v / FRAME_SCALE) for v in (top, right, bottom, left)]

        # --- DRAW VISUALS ---
        # Draw the bounding box
        cv2.rectangle(frame, (left, top), (right, bottom), color, 2)

        # Draw the label bar
        cv2.rectangle(frame, (left, bottom - 35), (right, bottom), color, cv2.FILLED)

        # Draw the name/status text
        font = cv2.FONT_HERSHEY_DUPLEX
        # Use contrasting text color (black for known, white for unknown alert)
        text_color = (0, 0, 0) if name != "Unknown" else (255, 255, 255)
        cv2.putText(frame, status_text, (left + 6, bottom - 6), font, 1.0, text_color, 1)

    # Display the resulting image
    cv2.imshow('Office Attendance Tracker', frame)

    # Hit 'q' on the keyboard to quit!
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

# Release handle to the webcam
video_capture.release()
cv2.destroyAllWindows()
```

3.2 Tracking and State Management

This phase focuses on logging movements and tracking the state (inside or outside) of the identified person. This is implemented by checking the last recorded action before logging the new, opposite action.

[LOGGED] 2025-10-21 14:49:44 - Neeraj has ENTRY.

[LOGGED] 2025-10-21 14:49:49 - Neeraj has EXIT.

3.3 Querying and Reporting

This final phase involves running specific queries against the persistent data log to answer management questions.

---

i> Configuration Consistency Check

The most important step for report_tool.py is to ensure its KNOWN_NAMES list matches the IDs used in the log file (attendance_app.py).

- **Required Names:** The KNOWN_NAMES list in report_tool.py must include **"Neeraj"** and the special identifier **"Unknown_Visitor"** (which is defined as UNKNOWN_VISITOR_ID in the configuration section).
- **Verification:** Ensure this section in your report_tool.py looks like this:

Python

```
# --- Configuration ---
LOG_FILE = 'attendance_log.csv'
UNKNOWN_VISITOR_ID = "Unknown_Visitor" # Must match the ID in attendance_app.py!
KNOWN_NAMES = ["Neeraj", "Yashith", UNKNOWN_VISITOR_ID] # CRITICAL: Includes all tracked IDs
```

ii> Execution and Usage

The report_tool.py script is executed directly from the command line and only works if attendance_app.py has run at least once to create the attendance_log.csv file.

You must run this script from the same folder where the log file is located.

Query Goal	**Command Example**
Current Status	python report_tool.py "who is currently in?"
Total Entries	python report_tool.py "Neeraj entry count today"
Unknown Summary	python report_tool.py "summary for unknown visitor"
Detailed Logs	python report_tool.py "show logs for Neeraj on 2025-10-21"
Command Flag	python report_tool.py who is in office

Sample python code file (report_tool.py) :

```
from itertools import count
import pandas as pd
from datetime import datetime
import argparse
from dateutil.parser import parse, ParserError
from typing import Optional, List, Tuple
import sys

# --- Configuration ---
LOG_FILE = 'attendance_log.csv'
```

```
UNKNOWN_VISITOR_ID = "Unknown_Visitor" # Must match the ID in attendance_app.py!
# IMPORTANT: Add the UNKNOWN_VISITOR_ID so the parser recognizes it
KNOWN_NAMES = ["Neeraj", "Yashith", UNKNOWN_VISITOR_ID]

# --- Helper Functions ---

def load_and_prepare_log() -> Optional[pd.DataFrame]:
    """Loads the CSV log and converts Timestamps to datetime objects."""
    try:
        # Suppress DtypeWarning for mixed types if columns are empty
        df = pd.read_csv(LOG_FILE, dtype={'Person_ID': str, 'Action': str})

        # Check if the log is empty (only headers exist)
        if df.empty or len(df) == 0:
            return pd.DataFrame(columns=['Person_ID', 'Action', 'Timestamp'])

        # Convert Timestamp column
        df['Timestamp'] = pd.to_datetime(df['Timestamp'])
        return df
    except FileNotFoundError:
        print(f"Error: Log file '{LOG_FILE}' not found. Run attendance_app.py first.")
        return None
    except Exception as e:
        print(f"An error occurred loading the log: {e}")
        return None

def parse_natural_query(query: str) -> Tuple[Optional[str], Optional[str], str]:
    """
    Parses a natural language query string to extract Person, Date, and Action.
    Returns (person, date_str, action)
    """
    query = query.lower()

    # Initialize all variables
    person: Optional[str] = None
    date_str: Optional[str] = None
    action: str = 'EXIT' # Default action

    # 1. Extract Person ID
    for name in KNOWN_NAMES:
        if name.lower() in query:
            person = name
            break

    # CRITICAL: If the user queries "unknown" or "visitor", map it to the ID.
    if person is None and ("unknown" in query or "visitor" in query):
        person = UNKNOWN_VISITOR_ID
```

```
    # 2. Extract Date
    try:
        # dateutil tries to find a date (e.g., 'today', 'Oct 14th', '2025-10-14')
        parsed_date = parse(query, fuzzy=True, dayfirst=False).date()
        date_str = parsed_date.strftime('%Y-%m-%d')
    except ParserError:
        pass # date_str remains None
    except Exception:
        pass # Ignore other parsing issues

    # 3. Determine Action/Command (Expanded Vocabulary)
    if any(keyword in query for keyword in ["total count", "total events", "summary"]):
        action = 'SUMMARY' # <-- NEW SUMMARY ACTION

    elif any(keyword in query for keyword in ["what time", "when", "show logs", "list entries", "list
exits"]):
        action = 'LOGS'

    elif "status" in query or "in or out" in query or "who is where" in query or "where is" in query:
        action = 'STATUS'

    elif any(keyword in query for keyword in ["enter", "entry", "come in", "came", "arrive",
"arrival", "check in"]):
        action = 'ENTRY' # Action for COUNT

    elif any(keyword in query for keyword in ["left", "exit", "went out", "gone", "check out",
"leave"]):
        action = 'EXIT' # Action for COUNT

    return person, date_str, action

def query_current_status(df: pd.DataFrame):
    """Answers: Who is currently IN or OUT?"""
    print("\n✅ Query Result: Current Status of People:")

    if df.empty:
        print("No log data found.")
        return

    # Get the last action for each person
    latest_actions = df.sort_values(by='Timestamp').groupby('Person_ID').last()

    for index, row in latest_actions.iterrows():
        status = row['Action']
        current_location = "IN" if status == "ENTRY" else "OUT"
        print(f"- {index}: Currently {current_location} (Last seen: {row['Timestamp']})")

def query_total_employees(df: pd.DataFrame):
```

```
    """Answers: How many unique people are in the log?"""
    unique_count = df['Person_ID'].nunique()
    print(f"\n✅ Query Result: The attendance log contains data for {unique_count} unique
person/entity(s).")

    if unique_count > 0:
        names = df['Person_ID'].unique()
        print(f"Names found: {', '.join(names)}")

def query_status_filter(df: pd.DataFrame, required_status: str):
    """Answers: Who is IN or WHO is OUT."""

    if df.empty:
        print(f"\n✅ Query Result: No log data found.")
        return

    latest_actions = df.sort_values(by='Timestamp').groupby('Person_ID').last()
    required_action = "ENTRY" if required_status == "IN" else "EXIT"

    filtered_people = latest_actions[latest_actions['Action'] == required_action]

    names_list = filtered_people.index.tolist()
    count = len(names_list)

    print(f"\n✅ Query Result: Currently, {count} person(s) are {required_status.upper()}:")

    if count > 0:
        print(f"People {required_status.upper()}: {', '.join(names_list)}")
    else:
        print(f"No people are currently {required_status.upper()}.")

def query_action_count(df: pd.DataFrame, person_id: str, action_type: str, date_str: Optional[str]
= None):
    """
    Generalized function to count ENTRY or EXIT events and format the output dynamically.
    """

    # 1. Filter the DataFrame based on Person and Action
    filtered_df = df[(df['Person_ID'] == person_id) & (df['Action'] == action_type)].copy()

    # 2. Filter by Date if provided
    if date_str:
        try:
            target_date = datetime.strptime(date_str, '%Y-%m-%d').date()
            filtered_df = filtered_df[filtered_df['Timestamp'].dt.date == target_date]
        except ValueError:
            print("Error: Invalid date format. Please use YYYY-MM-DD.")
            return
```

```
    count = len(filtered_df)

    # 3. Determine the natural date context for the sentence
    if date_str:
        date_context = f" on {date_str}"
    else:
        date_context = " in total"

    # 4. Determine the natural action phrase and create the dynamic response
    action_phrase = "came to the office" if action_type == "ENTRY" else "left the office"

    print(f"✅ Query Result: {person_id} {action_phrase} {count} time(s){date_context}.")

def query_specific_logs(df: pd.DataFrame, person_id: str, date_str: Optional[str] = None):
    """
    Retrieves and displays the actual timestamps/logs for a person.
    """

    # 1. Filter by Person
    filtered_df = df[df['Person_ID'] == person_id].copy()

    # 2. Filter by Date if provided
    date_context = " (All Time)"
    if date_str:
        try:
            target_date = datetime.strptime(date_str, '%Y-%m-%d').date()
            filtered_df['Date'] = filtered_df['Timestamp'].dt.date
            filtered_df = filtered_df[filtered_df['Date'] == target_date]
            date_context = f" on {date_str}"
        except ValueError:
            print("Error: Invalid date format.")
            return

    if filtered_df.empty:
        print(f"\n✅ Query Result: No log entries found for {person_id}{date_context}.")
        return

    print(f"\n✅ Query Result: Log entries found for {person_id}{date_context}:")

    # Sort and display the results
    filtered_df = filtered_df.sort_values(by='Timestamp')

    for index, row in filtered_df.iterrows():
        action = row['Action']
        time_only = row['Timestamp'].strftime('%H:%M:%S')
        date_only = row['Timestamp'].strftime('%Y-%m-%d')
```

```
        print(f"- {date_only} | {action:<7} recorded at {time_only}")

# --- MAIN EXECUTION ---

if __name__ == "__main__":
    df_log = load_and_prepare_log()

    if df_log is None:
        sys.exit(1)

    parser = argparse.ArgumentParser(description="Query the office attendance log using natural language or commands.")

    # Add new specific command-line flags for summary queries
    parser.add_argument('--total-employees', action='store_true', help='Show the total count and list of unique employees in the log.')
    parser.add_argument('--who-is-in', action='store_true', help='Show the list of employees currently IN the office.')
    parser.add_argument('--who-is-out', action='store_true', help='Show the list of employees currently OUT of the office.')

    # Argument for Natural Queries
    parser.add_argument('query_string', nargs='*', help='The full query, e.g., "status", "Neeraj left today", or "who is in?".')

    args = parser.parse_args()

    # --- HANDLE NEW COMMAND-LINE QUERIES FIRST ---
    if args.total_employees:
        query_total_employees(df_log)
    elif args.who_is_in:
        query_status_filter(df_log, "IN")
    elif args.who_is_out:
        query_status_filter(df_log, "OUT")

    # --- FALLBACK TO NATURAL LANGUAGE QUERY ---
    else:
        full_query = " ".join(args.query_string).strip()

        if not full_query:
            print("Please provide a query or use a command-line flag (e.g., 'status', 'Neeraj left today', or '--who-is-in').")
            parser.print_help()
            sys.exit(1)

        # Parse the natural query
        person, date_str, action = parse_natural_query(full_query)
```

```
    # --- EXECUTE PARSED NATURAL QUERY ---
    if action == 'STATUS':
        query_current_status(df_log)
    elif action == 'LOGS':
        if person:
            print(f"Interpreting query: Person={person}, Action=LOGS, Date={date_str if date_str else 'All Time'}")
            query_specific_logs(df_log, person, date_str)
        else:
            print("\nError: Please specify the person's name for this query (e.g., 'show logs for Neeraj').")

    elif action == 'SUMMARY':
        if person:
            # Determine the date context for the header
            date_context_header = f" on {date_str}" if date_str else " in total"

            print(f"Interpreting query: Person={person}, Action=SUMMARY, Date={date_str if date_str else 'All Time'}")
            print(f"\n--- Attendance Summary for {person}{date_context_header} ---")

            # Call query_action_count for ENTRY
            query_action_count(df_log, person, 'ENTRY', date_str)

            # Call query_action_count for EXIT
            query_action_count(df_log, person, 'EXIT', date_str)
            print("-----------------------------------------------------------------")
        else:
            print("\nError: Please specify the person's name for a summary query.")

    elif person and action in ['ENTRY', 'EXIT']:
        # This is the original, specific COUNT query
        print(f"Interpreting query: Person={person}, Action={action}, Date={date_str if date_str else 'All Time'}")
        query_action_count(df_log, person, action, date_str)
    else:
        print(f"\nCould not fully understand query or missing person/action: '{full_query}'")
        print("Please ensure the query contains a person's name or a recognized status command.")
```

---

4. Implementation Steps

The implementation requires the following steps:

1. **Create a Known Faces Database:** Generate and save encodings for the three employees (or more) in a persistent structure (e.g., a dictionary or array).

2. **Recognition Loop:** In the video processing script, for every detected face, iterate through the known encodings to find a match. This
   uses face_recognition.compare_faces and face_recognition.face_distance.
3. **Log Function:** Write a function to append the person's ID, the current time, and the determined action (**ENTRY/EXIT**) to a persistent file (CSV).
4. **Reporting:** Write separate functions to read and analyze the CSV file to answer specific questions.

---

5. Output

The system must deliver the following core outputs:

- **Trained Model Weights:** Face encodings database for known personnel.
- **Data Log:** A continuously updated CSV file of all ENTRY and EXIT events, adhering to the defined schema.
- **Query Capabilities:** The ability to run logic on the data log to provide answers to complex queries, including:
    - **Action Count:** Counting all rows where Person ID = 'Neeraj' AND Action = 'EXIT' AND Date = 'Today'.
    - **Current Status:** Finding the latest log entry for each person and filtering for people whose latest Action is 'EXIT' ("Who is currently NOT in the office?").
    - **Duration:** Finding all ENTRY and EXIT pairs for a person on a specific day, calculating the time difference, and summing them up ("How long was Yashith in the office on Monday?").

---

6. References

- **Project Text:** Intelligent Vision System for Attendance and Access Monitoring Requirements.
- **Core Technology:** Person Re-Identification (ReID).
- **Libraries:** face_recognition, opencv, pandas, datetime, argparse.